

How to Do the Research, Keep a Logbook, and Store Results

A beginner's guide for the Sarcoma QML Benchmark project. Written 2026-09-22. It uses this project's real files and real findings as the examples.

1. The big picture: how a research paper actually happens

A paper is the last step, not the first. Most of the work is careful record-keeping so that a stranger (or you, six months from now) can see exactly what was done and why. The main rule from a well-known guide on organizing computational projects is: **someone unfamiliar with your project should be able to look at your files and understand in detail what you did and why.**

Stage	What you do	Where you are now
1. Question and hypothesis	Write the question and the null hypothesis before touching results.	Done (spec sections 1 and 7).
2. Read the field	Find what is known, and where the gap is.	Started (two reference lists).
3. Protocol (the plan)	Write down exactly what you will run and how you will judge it.	Mostly done (spec.md).
4. Get and check the data	Download, archive, verify counts.	Steps 1-5 done. Cohort size decision still open.
5. Run experiments	Run the pipeline, save everything.	Not started.
6. Analyze	Statistics, tables, figures.	Not started.
7. Write and share	Turn the logbook into a paper.	Later.

Why the order matters: if you decide what to test after seeing results, you can fool yourself without meaning to. Your spec already handles this well: it says to write the null hypothesis into the record before training starts (step 15) and to report a null result as seriously as a positive one.

2. The logbook

A logbook is a dated diary of your research. It is not a report. It is where you write what you did, what you saw, and why you made each choice, at the time you do it. Later, your paper's Methods section is mostly copied from it.

The rules

- **Date every entry.** Newest at the bottom. One file per day or per work session.
- **Never delete or rewrite an old entry.** If you were wrong, add a new dated correction and leave the old text visible. (This project already did this: the uterine-LMS mistake is corrected with a dated note, not erased.)
- **Write the number, not the feeling.** "98.1% complete (152 of 155)" beats "looks mostly complete".

- **Record the command or script** that produced each number, so you can repeat it.
- **Write down decisions and the reason.** A choice with no reason cannot be defended to a reviewer.
- **Record failures and dead ends.** The missing mutation file and the wrong subtype counts were both useful findings. Negative results belong in the log.
- **Write it the same day.** Memory fades fast.

A daily entry template

```
DATE: 2026-09-22
GOAL:    What I was trying to find out today (one sentence).
DID:     What I ran. Script names / commands.
SAW:     The actual numbers and outputs.
DECIDED: The choice I made, and WHY.
PROBLEMS: Anything that broke, surprised me, or looks wrong.
OPEN:    Questions still unanswered.
NEXT:    The very next step.
```

A real example from this project

```
DATE: 2026-09-22
GOAL:    Decide whether clinical columns (file 1) should be model features.
DID:     Compared each column with the LMS/DDLPs label on the 155-patient
          cohort (Python, standard library only).
SAW:     Tumor site alone ~70% vs 62.6% always-guess-LMS baseline (in-sample).
          About 28 of 97 LMS have uterine site codes; 0 of 58 DDLPs.
          7 staging-type columns are 100% blank.
DECIDED: Keep omics-only as the main experiment. WHY: site and sex are
          shortcuts (tissue of origin), not tumor biology.
PROBLEMS: My earlier note said uterine LMS were out of scope. WRONG.
          Added a dated correction; old text left visible.
OPEN:    Confirm uterine count with TUMOR_TISSUE_SITE. Decide on a
          clinical-only baseline and a no-uterine sensitivity run.
NEXT:    Get the user's decision, then update the spec.
```

Keep three kinds of notes

Note type	What it holds
Logbook	Chronological diary of what you did (above).
Decision / deviation log	A short list of every place you departed from the plan, with the reason. Example: "Used the API for mutations because the file was missing." Reviewers ask about these.
Pre-registration note	The hypothesis and the single main test, written and dated BEFORE training. Never edited afterwards.

You already have a good start: `NOTES.md` in the raw-data folder is a real logbook. As the project grows, move to a `logbook/` folder with one dated file per session.

3. Storing results

The goal: any number in your paper can be traced back to a run, a script, a setting, and a dataset version. Suggested layout for this project:

```
sarcoma-qml-benchmark/  
  spec.md                the plan (edit only with a dated note)  
  logbook/              dated diary files, one per session  
  decisions.md           deviations from the plan, with reasons  
  data/  
    raw/2026-09-13_cbioportal/ as downloaded. READ-ONLY. has CHECKSUMS  
    processed/           anything you generate from raw  
  configs/              one settings file per experiment (seeds!)  
  src/                  all code (put it under git)  
  results/  
    exp1_full_cohort/  
      2026-10-01_mlp_seed42/ <- one folder per run, never overwritten  
        config.yaml         every setting used, incl. random seeds  
        git_commit.txt      exact code version  
        env.txt             library versions  
        fold_metrics.csv    one row per fold and model  
        predictions.csv     per-patient, per-fold scores  
        selected_features.json genes chosen in each fold  
        run_log.txt         what the program printed  
    exp2_ablation/  
    exp3_noise_sim/  
  figures/              made only from files in results/  
  paper/               the manuscript
```

What to save from every run

- **Per-patient predictions, not just summary scores.** With predictions saved you can recompute any metric later (PR-AUC, MCC, anything) and run any statistical test without retraining.
- **Per-fold numbers.** Your significance test works on the individual fold results, not on the average.
- **The settings and seeds.** Your spec already requires fixing and logging seeds for network start values, circuit start values, and fold assignment.
- **Which genes were selected in each fold.** The spec asks for this too.
- **Code version and library versions.** Use git commits, and save package versions.

An example results table (fold_metrics.csv)

```
run_id,model,repeat,fold,n_train,n_test,n_features,seed,pr_auc,roc_auc,bal_acc,mcc  
exp1_mlp_seed42,mlp,0,0,124,31,8,42,0.81,0.88,0.76,0.52  
exp1_mlp_seed42,mlp,0,1,124,31,8,42,0.77,0.85,0.72,0.47  
(illustrative numbers only, not real results)
```

Habits that prevent disasters

- **Never overwrite.** New run, new folder. Disk space is cheaper than a lost result.
- **Raw data is read-only.** Work only on copies in processed/.
- **Figures come from saved files.** A figure must be re-creatable by running a script on files in results/, with no manual editing.
- **Back up.** Keep a second copy (cloud drive or external disk). Put code on a git host.

- **Keep the failed runs** in a clearly named folder. You may need to report them.

4. Writing the paper: what goes where

Use the standard structure (Introduction, Methods, Results, Discussion). Below is what each part holds for *this* project.

Section	What goes in it (for this project)
Abstract, Title	Write these last. Two or three sentences each on: problem, method, main result, meaning.
Introduction	The problem (very small patient counts, huge numbers of genes). What quantum machine learning claims. The gap: are the claims true against fair classical baselines? Your question and your pre-stated null hypothesis.
Methods	Cohort and where the 105/58 counts came from (the 2016 Firehose freeze and the 5 missing patients). Data sources. Preprocessing done inside each fold. The models. The cross-validation design. Metrics. The one confirmatory test and the correction for the rest. Software versions. Code availability.
Results	Experiment 1 (all patients). Experiment 2 (shrinking training size). Experiment 3 (simulated noise, state clearly it is a simulation). Clinical-only baseline. Sensitivity run without uterine LMS.
Discussion	What the results mean. Report a null result honestly if that is what you get. Compare with prior work.
Limitations	Small N. One cohort (plus the required second-cohort check). Simulated noise, not hardware. The uterine/tissue confound. The 8-16 qubit ceiling. Choices left open, such as the encoding circuit.
Data and code availability	Where the data came from (public) and where your code lives.

Practical advice

- **Tell one story.** Decide the single main claim. Every figure and table should support it. (The 'Ten simple rules for structuring papers' article below explains this well.)
- **Make the figures and tables first**, then write the text around them.
- **Write Methods while you work**, copying from the logbook. It is the easiest section if you kept good notes.
- **Say what you did not do.** Reviewers trust honest limits more than big claims.
- **Use a reporting checklist.** For papers about prediction models, TRIPOD+AI is a checklist of what to report. Skim it now so you record the right things while you work.
- **Cite peer-reviewed sources** for every method choice. A YouTube video helps you understand; it is not what you cite.
- **Keep your references in a manager** (for example Zotero) from day one, not at the end.

5. Where to learn preprocessing: papers or YouTube?

Short answer: both, for different jobs. Neither alone is enough.

Source type	Good for	Weak at
YouTube videos	Fast intuition. Seeing an idea explained simply.	Quality varies. Not something you can cite. Rarely covers your exact data type.
Books and courses	Step-by-step learning with exercises. Best for fundamentals.	Slower. Still general, not specific to TCGA files.
Software documentation	The correct way to write the code (for example avoiding leakage).	Tells you how, not why.
Research papers	The reason behind each choice, and what you cite in your paper.	Written for experts. Poor for learning from zero.

The papers I gave you earlier are the **why and the citations**, not tutorials. Learn the idea first from a video or book, then read the paper to see the rigorous version and to cite it. Finally, **re-implement it yourself**: that is where real learning happens.

A suggested learning order for this kind of data

Skill	Learn it from	Go deeper / cite
1. Core ideas: cross-validation, regularization (ElasticNet), overfitting	The StatQuest with Josh Starmer channel for intuition. Search it for "Elastic Net" and "cross validation" (I confirmed videos on both exist; I have not watched them).	The free book An Introduction to Statistical Learning (Python edition) .
2. Doing it in code without cheating (data leakage, pipelines)	scikit-learn: Common pitfalls and recommended practices . Key rule: never fit any step on test data.	Lones: How to avoid machine learning pitfalls ; Kapoor and Narayanan on leakage .
3. Gene-expression and high-dimensional data specifically	HarvardX Data Analysis for the Life Sciences (free book and courses; includes a High-Dimensional Data Analysis course).	The papers in the earlier reference list (independent filtering, ElasticNet, selection bias).
4. Your specific file formats (cBioPortal, GDC, TCGA)	The cBioPortal and GDC documentation pages. (I did not verify the exact pages; search their docs for "file formats".)	The TCGA sarcoma paper and the 'about your files' section of the earlier PDF.

Practice tip: before touching the sarcoma data, build the whole pipeline (scaling, feature selection, cross-validation, metrics) on a small practice dataset, for example scikit-learn's built-in breast-cancer dataset. Then plug in the real data. Mistakes are cheap on practice data and expensive on your real experiment.

6. A simple weekly rhythm

- **Every session:** write a logbook entry before you stop.
- **Every week:** reread the week's entries, update the decisions log, and check that every result you want to keep is saved in its own run folder.
- **Before each new experiment:** re-read the plan (spec.md) and confirm you are not changing the question after seeing results.
- **Before writing:** list every claim you want to make, and next to each write the run folder that proves it.

7. Links (all confirmed to exist via search)

Organizing and record-keeping

- [Noble 2009: A Quick Guide to Organizing Computational Biology Projects](#)
- [Sandve et al. 2013: Ten Simple Rules for Reproducible Computational Research](#)
- [Schnell 2015: Ten Simple Rules for a Computational Biologist's Laboratory Notebook](#)

Writing and reporting

- [Mensh and Kording 2017: Ten simple rules for structuring papers](#)
- [TRIPOD+AI \(2024\): reporting guidance for prediction models that use machine learning](#)

Machine learning pitfalls and learning resources

- [Lones: How to avoid machine learning pitfalls: a guide for academic researchers](#)
- [Kapoor and Narayanan 2023: Leakage and the reproducibility crisis in ML-based science](#)
- [scikit-learn: Common pitfalls and recommended practices](#)
- [An Introduction to Statistical Learning \(free, includes Python edition\)](#)
- [HarvardX: Data Analysis for the Life Sciences](#)
- [StatQuest with Josh Starmer \(YouTube channel\)](#)

Note: I confirmed each resource exists and read its description in search results. I have not read every article in full or watched the videos, so skim each one before relying on it.